

Development and Evaluation of Various Optimization Techniques for Multi-Robot Task Allocation and Routing in Warehouse Settings

Rana M. Talaat¹, Sama Tamer¹, Menna Walaa², Aya Walid¹, Dalia M. Mahfouz^{2,3}  and Omar M. Shehata^{2,3} 

Abstract—The coordination of autonomous mobile robots (AMRs) in modern warehouses presents a complex multi-objective optimization problem involving task allocation, routing efficiency, collision avoidance, and energy management. As warehouse systems scale, many existing optimization approaches become computationally expensive or require extensive parameter tuning. This paper proposes a Teaching–Learning–Based Optimization (TLBO) framework for multi-robot task allocation and routing in structured warehouse environments. The proposed method integrates task assignment, robot routing, battery-aware operation, and collision avoidance within a unified optimization formulation. The TLBO algorithm is evaluated through simulation in three warehouse instances of increasing complexity. Its performance is compared with Genetic Algorithm (GA), Simulated Annealing (SA), and Particle Swarm Optimization (PSO) under identical modeling assumptions. Experimental results demonstrate that TLBO achieves stable convergence behavior and consistently produces feasible collision-free solutions across all warehouse scenarios while eliminating the need for algorithm-specific parameter tuning. The results indicate that TLBO provides a robust and scalable optimization framework for energy-aware multi-robot warehouse coordination.

Index Terms—Warehouse robotics, multi-robot systems, task allocation, routing optimization, Teaching–Learning–Based Optimization, metaheuristics, TLBO.

I. INTRODUCTION

The rapid growth of e-commerce and on-demand fulfillment services has significantly increased operational pressure on modern warehouse systems. Distribution centers are required to process large volumes of orders within strict delivery deadlines, often under lean inventory policies inspired by Just-In-Time (JIT) logistics principles, which reduce warehouse management costs but increase the complexity of robot coordination.

Autonomous Mobile Robots (AMRs) have become a central component of modern warehouse automation systems. Early large-scale deployments demonstrated the feasibility of coordinating a significant number of robots [1], [2]. However, as the number of robots increases, the complexity of multi-robot task allocation and routing grows combinatorially.

Efficient robot coordination is essential to ensure successful task completion, minimize travel time, and prevent excessive battery depletion.

In addition, modern warehouses increasingly operate under shared human robot collaboration (HRC) settings, where strict safety and predictability requirements must be maintained [1]. Energy management further complicates the optimization process, as battery limitations and charging logistics directly impact robots' availability and throughput efficiency [3].

Consequently, multi-robot warehouse optimization will remain a constrained, multi-objective, NP-hard optimization problem integrating assignment, routing, energy management, and safety requirements. Designing scalable and robust optimization frameworks capable of handling these interdependent constraints remains a major research challenge [3].

A. Related Work and Research Gap

Existing work on multi-robot warehouse coordination can be grouped into three main directions. First, review and coordination studies discuss the importance of scalable task allocation and safe robot coordination in automated warehouses [1]. Second, exact and integrated optimization methods can solve task assignment and routing problems effectively, but they become costly as the number of robots and tasks increases [2], [4]. Third, metaheuristic methods such as GA, SA, and PSO improve scalability, but their performance depends on parameters such as mutation rates, cooling schedules, inertia weights, and acceleration coefficients [5]–[7].

Despite these advances, many approaches still treat task allocation, routing, energy management, and collision avoidance separately, or require extensive parameter tuning. This creates a need for a unified and parameter-light optimization framework. Therefore, this paper investigates TLBO for energy-aware multi-robot task allocation and routing under battery and collision constraints [8], [9].

B. Proposed Approach and Contributions

This paper proposes a Teaching–Learning–Based Optimization (TLBO) framework for energy-aware multi-robot task allocation and routing in a structured warehouse environment. Unlike GA and PSO algorithms, TLBO does not use any specific control parameters such as mutation rates and inertia weights, making it more robust and less dependent on parameter tuning [8].

The contributions of this paper are summarized as follows:

- A unified multi-objective formulation integrating task allocation, routing efficiency, battery scheduling, robot activation, and strict collision avoidance constraints.
- An adaptation of the TLBO to handle mixed discrete and continuous decision variables representing assignment and sequencing decisions.

¹Faculty of Media Engineering and Technology (MET), German University in Cairo, Egypt, {rana.ahmed, sama.abdelrahman, aya.kenawy}@student.guc.edu.eg

²Mechatronics Dept., Faculty of Engineering and Materials Science (EMS), German University in Cairo, Egypt, {mennatallah.osama}@student.guc.edu.eg, {dalia.mahfouz, omar.mohamad}@guc.edu.eg

³Multi-Robot Systems (MRS) Research Group, Cairo, Egypt

- A scalable evaluation across three warehouse case studies of increasing complexity:
 - Small: 30×30 warehouse, 2 robots, 10 tasks.
 - Medium: 40×40 warehouse, 3 robots, 15 tasks.
 - Large: 50×50 warehouse, 5 robots, 20 tasks.
- Comparison of the proposed method with GA, SA, and PSO algorithms under similar modeling and feasibility considerations.

While the performance of different algorithms may differ depending on the scale of the problem, the proposed TLBO-based algorithm ensures the stable convergence behavior, parameter-light operation, and consistent feasibility under battery and collision constraints, making it suitable for practical warehouse deployment scenarios.

C. Paper Organization

The remainder of this paper is structured as follows. Section II discusses the problem formulation, including decision variables, objective functions, and operational constraints. Section III describes the TLBO-based optimization methodology and implementation details. Section IV presents the results and the comparative study. Finally, Section V concludes the paper and outlines future research directions.

II. PROBLEM FORMULATION

This work considers a multi-robot warehouse coordination problem in which a fleet of autonomous robots executes pickup and delivery tasks while minimizing operational cost and respecting physical and energy constraints. The warehouse is represented as a discrete grid environment $\mathcal{W} \subset \mathbb{Z}^2$ of size $W_x \times W_y$. A set of robots $\mathcal{R} = \{1, \dots, R\}$ must complete a set of tasks $\mathcal{T} = \{1, \dots, T\}$.

Each task $k \in \mathcal{T}$ consists of a pickup location $P_k = (x_k^p, y_k^p)$ and a delivery location $D_k = (x_k^d, y_k^d)$. Robots start and finish their routes at a depot D_0 . During operation, robots must respect collision avoidance constraints, battery limitations, and task precedence rules. When the battery level of a robot falls below a predefined threshold, the robot must temporarily interrupt its task execution and navigate to the nearest charging station.

The goal of the optimization is to determine task assignments and execution sequences that minimize operational cost while maintaining safe and feasible robot coordination.

A. Optimization Model

The global objective is defined using a weighted normalized fitness function:

$$F = w_1 \frac{C_{\max}}{100} + w_2 \frac{D_{\text{total}}}{500} + w_3 \frac{R_{\text{used}}}{R} + w_4 \frac{B_{\text{cons}}}{500} - w_5 \frac{T_{\max}}{T} + w_6 \frac{S_{\text{imb}}}{100} + w_{\text{viol}} V. \quad (1)$$

where $C_{\max}$ is the makespan, D_{total} is the total travel distance, R_{used} is the number of active robots, R is the total number of available robots, B_{cons} is the total battery consumption, $T_{\max}$ is the maximum number of tasks assigned

to one robot, T is the total number of tasks, S_{imb} is the workload imbalance/smoothing index, and V is the total constraint violation penalty accounting for collisions, battery violations, and infeasible task orders.

The weights were set to $w_1 = 0.15$, $w_2 = 0.10$, $w_3 = 0.15$, $w_4 = 0.05$, $w_5 = 0.05$, $w_6 = 0.10$, and $w_{\text{viol}} = 20$. These values prioritize feasible and balanced robot coordination while still considering makespan, travel distance, and energy consumption. The normalization constants 100, 500, 500, and 100 were used for makespan, total distance, battery consumption, and workload imbalance, respectively, so that no metric dominates the objective only because of its numerical scale.

B. Motion and Energy Model

Robot travel distance between two grid locations A and B is computed using Manhattan distance

$$d(A, B) = |x_A - x_B| + |y_A - y_B|. \quad (2)$$

Battery consumption is proportional to the traveled distance, while charging increases the battery level at a constant rate. Each robot has a battery capacity of 100%, and when the battery level drops below a threshold b_{th} which is 20%, the robot is required to navigate to the nearest charging station before continuing task execution.

C. Operational Constraints

The optimization is subject to the following constraints:

- **Task assignment:** each task must be assigned to exactly one robot.
- **Pickup–delivery precedence:** pickup must occur before the corresponding delivery.
- **Collision avoidance:** robots must maintain a minimum separation distance during motion.
- **Battery feasibility:** robot battery levels must remain non-negative, and charging is triggered when the battery falls below the predefined threshold.
- **Capacity limits:** each robot can only execute a limited number of tasks.

The optimization problem is a mixed discrete–continuous, multi-objective NP-hard problem that integrates task allocation, routing, and energy-aware robot coordination.

III. METHODOLOGY

This section presents the metaheuristic optimization framework adopted in this work. Simulated Annealing (SA), Genetic Algorithm (GA), and Particle Swarm Optimization (PSO) are used as baseline methods.

Teaching–Learning-Based Optimization (TLBO) constitutes the primary proposed approach. All methods share a unified decoding and evaluation pipeline: (i) decode a candidate solution into task-to-robot assignment and per-robot execution order, (ii) simulate robot motion on the grid, (iii) update battery with a 20% charging trigger, (iv) detect collisions using minimum separation constraints, and (v) compute the

TABLE I: Parameter settings used for the compared optimization algorithms.

Algorithm	Parameter settings
GA	Population size = 50, generations = 10, mutation probability = 0.1, single-point crossover, elitism-based population update.
SA	Initial temperature = 100, cooling rate = 0.95, iterations = 10, Metropolis acceptance rule.
PSO	Swarm size = 50, iterations = 10, inertia weight $w = 0.7$, $c_1 = 1.5$, $c_2 = 1.5$.
TLBO	Population size = 80 for small and medium instances, 100 for the large instance, maximum iterations = 150, teaching factor $T_f \in \{1, 2\}$.

aggregated fitness with constraint-aware penalties. This unified evaluation ensures fair comparison across methods. Table I summarizes the main parameter settings used for the comparative algorithms. These values were kept fixed across the warehouse instances unless otherwise stated.

A. Simulated Annealing (SA)

Simulated Annealing is a stochastic optimization algorithm inspired by the annealing process in metallurgy [6]. Candidate solutions are generated by local perturbations (swap/reassign operators) and accepted using the Metropolis rule:

$$P(\Delta F) = \exp\left(-\frac{\Delta F}{T}\right), \quad (3)$$

where ΔF is the fitness difference and T is the temperature.

Algorithm 1 Simulated Annealing

- 1: Initialize solution X and temperature T
- 2: **while** stopping criterion not met **do**
- 3: Generate neighbor X'
- 4: Compute $\Delta F = F(X') - F(X)$
- 5: **if** $\Delta F < 0$ or $\text{rand} < e^{-\Delta F/T}$ **then**
- 6: $X \leftarrow X'$
- 7: **end if**
- 8: Decrease T
- 9: **end while**

Collision avoidance and battery constraints are handled via penalty augmentation in the fitness evaluation; charging events are explicitly simulated when the battery level drops below 20%.

B. Genetic Algorithm (GA)

Genetic Algorithm is an evolutionary optimization method based on natural selection [5]. A population evolves through selection, crossover, and mutation.

Algorithm 2 Genetic Algorithm

- 1: Initialize population of chromosomes
- 2: **while** generation limit not reached **do**
- 3: Evaluate fitness of all chromosomes
- 4: Select parents based on fitness
- 5: Apply crossover and mutation
- 6: Repair / penalize infeasible individuals
- 7: Update population (elitism)
- 8: **end while**

In the implementation, chromosomes encode both task assignment and sequencing. Collision avoidance is evaluated using fine time-resolution checks, while infeasible individuals are handled through repair and constraint-priority fitness penalties.

C. Particle Swarm Optimization (PSO)

Particle Swarm Optimization is a population-based algorithm inspired by collective swarm behavior [7]. Each particle updates velocity and position:

$$v_i^{t+1} = wv_i^t + c_1r_1(p_i^{best} - x_i^t) + c_2r_2(g^{best} - x_i^t), \quad (4)$$

$$x_i^{t+1} = x_i^t + v_i^{t+1}, \quad (5)$$

where x_i^t and v_i^t are the position and velocity of particle i at iteration t , p_i^{best} and g^{best} are the personal and global best positions, w is the inertia weight, c_1 and c_2 are the cognitive and social coefficients, and $r_1, r_2 \in (0, 1)$ are random values.

Algorithm 3 Particle Swarm Optimization

- 1: Initialize particles (positions, velocities)
- 2: **while** iteration limit not reached **do**
- 3: Evaluate fitness; update p^{best} and g^{best}
- 4: Update velocity and position using fixed inertia weight w
- 5: Decode continuous particle to discrete plan
- 6: Repair / penalize infeasible particles
- 7: **end while**

Decoding maps continuous vectors to discrete assignments and priorities, while constraint violations (collision/battery) are handled via repair and penalties.

D. Proposed Teaching–Learning–Based Optimization (TLBO)

TLBO is a parameter-light population-based metaheuristic inspired by classroom teaching and peer learning [8], [9]. Each learner is represented as a continuous vector that is decoded into task assignments and sequencing priorities. TLBO proceeds in two phases.

Teacher phase: the best learner is selected as the teacher and guides the population toward improved mean learning:

$$X_{\text{new}} = X_{\text{old}} + r(X_{\text{teacher}} - T_f \bar{X}) \quad (6)$$

Learner phase: learners interact pairwise to exchange information:

$$X_i^{\text{new}} = \begin{cases} X_i + r(X_i - X_j), & F_i < F_j, \\ X_i + r(X_j - X_i), & \text{otherwise.} \end{cases} \quad (7)$$

where X_i and X_j are learner solutions, X_{teacher} is the best learner, $\bar{X}$ is the population mean, $T_f \in \{1, 2\}$ is the teaching factor, F_i and F_j are fitness values, and $r \in (0, 1)$ is a random value.

Algorithm 4 Teaching–Learning–Based Optimization

```

1: Initialize learner population
2: while maximum iterations not reached do
3:   Evaluate fitness of all learners
4:   Select teacher (best learner), compute mean  $\bar{X}$ 
5:   for each learner do
6:     Teacher update; accept if fitness improves
7:   end for
8:   for each learner do
9:     Select random peer; learner update; accept if im-
       proves
10:  end for
11: end while

```

Each learner vector is decoded into task assignments and execution priorities. Battery drain, 20% charging triggers, and collision checks are simulated during route execution. The best fitness value at each iteration is recorded to generate convergence curves.

IV. RESULTS AND DISCUSSION

A. Simulation Environment

Three warehouse instances of increasing scale were considered to evaluate algorithm performance and scalability:

- **Small instance:** 10 tasks, 2 robots, 30×30 grid.
- **Medium instance:** 15 tasks, 3 robots, 40×40 grid.
- **Large instance:** 20 tasks, 5 robots, 50×50 grid.

Each scenario includes a depot, charging station(s), pickup locations, and delivery nodes. Robots must respect battery constraints with mandatory charging at 20% threshold, and all solutions are required to remain collision-free. Fig. 1 illustrates the TLBO-generated routing plans for the three warehouse instances.

A supplementary video demonstrating the MATLAB-based simulation and the dynamic execution of the TLBO-generated coordination strategy is available online at: <https://youtu.be/gS9d2WPDZGo>.

B. Performance Metrics and Computational Cost

Table II summarizes the final fitness, makespan, total distance, active robots, collision feasibility, and normalized runtime. In the small instance, TLBO achieves the best fitness and lowest travel distance while remaining collision-free. In the medium instance, PSO obtains the lowest fitness, followed by TLBO, while SA shows weaker performance under increased task density. In the large instance, SA achieves the lowest fitness, whereas TLBO remains feasible but activates only 4 out of 5 robots, indicating partial workload imbalance.

Across all instances, every algorithm produces collision-free solutions. TLBO requires moderately higher runtime due to its two-phase update mechanism, but it maintains stable feasibility without algorithm-specific tuning.

C. Makespan and Distance Comparison

Fig. 2 compares makespan and total travel distance across all instances. TLBO performs best in the small instance, PSO gives the strongest medium-instance makespan, and SA performs best in the large instance. The increasing differences across scenarios show that algorithm behavior becomes more scale-sensitive as warehouse complexity increases.

D. Convergence Analysis

Fig. 3 shows that TLBO converges smoothly across the tested instances. PSO converges rapidly but exhibits larger early fluctuations, while SA improves more gradually. This supports the stability of TLBO, especially in constraint-aware search settings.

E. Repeated-Run Fitness Distribution

Fig. 4 shows repeated-run fitness variation. GA has higher variability, especially in medium and large instances, while PSO shows more compact distributions. TLBO is shown as a reference point because repeated TLBO statistics are still under evaluation.

F. Discussion

Although performance varies across instances, the results highlight several structural advantages of the proposed TLBO-based framework.

First, TLBO demonstrates consistently stable convergence behavior across problem scales (Fig. 3). Unlike PSO, which exhibits large initial oscillations in medium and large instances, and SA, which may converge slowly under increased complexity, TLBO maintains controlled descent dynamics without requiring algorithm-specific tuning.

Second, TLBO operates without control parameters such as mutation rates, crossover probabilities, inertia weights, or cooling schedules. This parameter-light nature reduces sensitivity to hyperparameter selection and improves robustness under mixed discrete–continuous search spaces.

Third, TLBO maintains feasibility under strict battery and collision constraints across all evaluated instances. Even in the large-scale case, where only 4 out of 5 robots were activated, the solution remains collision-free and operationally valid, indicating that TLBO prioritizes constraint-consistent task allocation over forced load balancing.

While PSO achieves strong performance in the medium instance and SA performs competitively in the large instance, TLBO exhibits balanced behavior in terms of solution quality, convergence smoothness, and computational stability.

These characteristics make TLBO particularly suitable for warehouse environments where:

- The search space is highly combinatorial,
- Constraint interactions are nonlinear,
- Parameter tuning is impractical in real deployments.

Overall, the results support TLBO as a robust and scalable optimization framework for battery-aware, collision-constrained multi-robot warehouse coordination.

TLBO - Warehouse Layout & Robot Routes

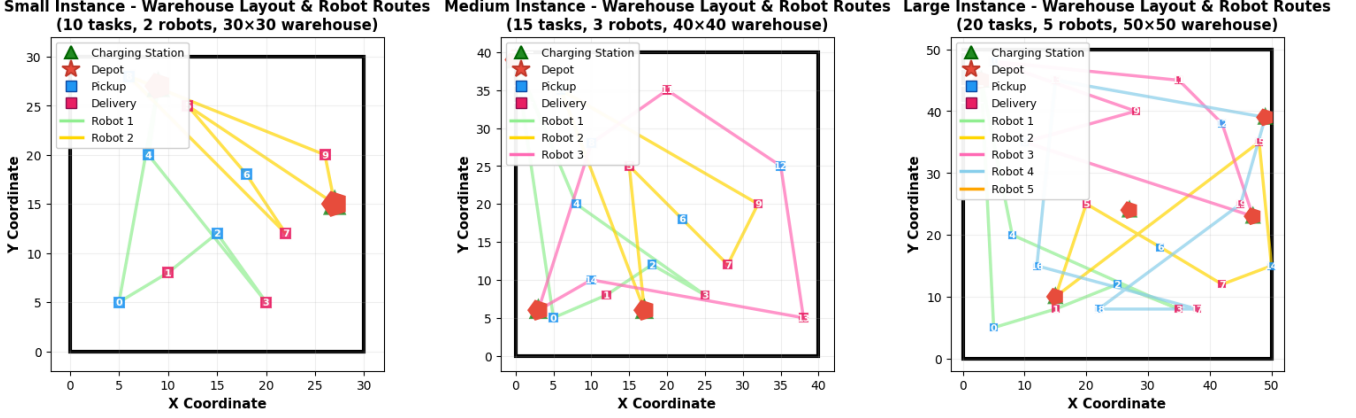


Fig. 1: Warehouse layouts and TLBO-generated robot routes for small, medium, and large instances.

TABLE II: Comparative performance results across all instances (fitness, operational metrics, feasibility, and normalized runtime).

Instance	Alg.	Fitness	Makespan	Total Distance	Active Robots	Collision-Free	Time/Run (norm.)
Small	GA	0.8530	202.93	478	2/2	Yes	0.02
Small	SA	0.3463	123.78	282	2/2	Yes	0.01
Small	PSO	0.3551	97.80	278	2/2	Yes	0.03
Small	TLBO	0.3445	95.90	252	2/2	Yes	0.11
Medium	GA	1.0000	526.68	1022	3/3	Yes	0.01
Medium	SA	4.7674	314.13	702	3/3	Yes	0.00
Medium	PSO	0.4872	127.10	544	3/3	Yes	0.03
Medium	TLBO	0.6092	191.79	606	3/3	Yes	0.08
Large	GA	1.0000	730.60	1774	5/5	Yes	0.02
Large	SA	0.7576	187.78	1062	5/5	Yes	0.00
Large	PSO	1.2321	271.55	1664	5/5	Yes	0.03
Large	TLBO	3.2982	460.76	1622	4/5	Yes	0.12

V. CONCLUSIONS AND FUTURE RECOMMENDATIONS

This paper investigated the multi-robot task allocation and routing problem in warehouse environments under battery and collision constraints. A Teaching–Learning–Based Optimization (TLBO) framework was proposed to jointly address task assignment, robot routing, energy management, and operational feasibility within a unified optimization model.

The proposed method was evaluated using three warehouse scenarios of increasing scale and compared with Genetic Algorithm (GA), Simulated Annealing (SA), and Particle Swarm Optimization (PSO).

The experimental results demonstrate that TLBO maintains stable convergence behavior and consistently produces feasible collision-free solutions while respecting battery constraints. Although other algorithms may achieve lower objective values in specific scenarios, TLBO provides balanced performance and improved robustness due to its parameter-light structure. Overall, the results highlight the potential of TLBO as a practical optimization technique for warehouse robot coordination, particularly in environments where parameter tuning is difficult and operational constraints must be strictly satisfied. Future work can extend the proposed framework in

several directions. First, the current work considers static task sets; therefore, incorporating dynamic task arrivals and real-time task reallocation would improve the applicability of the model in real warehouse operations. Second, larger warehouse environments with higher robot densities could be explored to further evaluate scalability. Hybrid optimization strategies combining TLBO with other metaheuristics or local search techniques may also enhance solution quality in large-scale instances. Finally, integrating more realistic robot motion models, traffic management mechanisms, and real-world robotic platforms would provide further validation of the proposed approach and support its deployment in industrial warehouses.

REFERENCES

- [1] Y. Li, H. Ma, and S. Koenig, “Multi-robot coordination and task allocation in automated warehouse systems: A review,” *Robotics and Autonomous Systems*, vol. 146, p. 103879, 2021.
- [2] Y. Qin, H. Liu, and Z. Zhang, “Integrated task assignment and path planning for multi-type robots in an intelligent warehouse system,” *Transportation Research Part E: Logistics and Transportation Review*, vol. 187, p. 103481, 2024.
- [3] Y. Zhang, H. Chen, and D. Liu, “A distributed multi-robot task allocation method for time-constrained dynamic collective transport,” *Robotics and Autonomous Systems*, vol. 175, p. 104601, 2024.

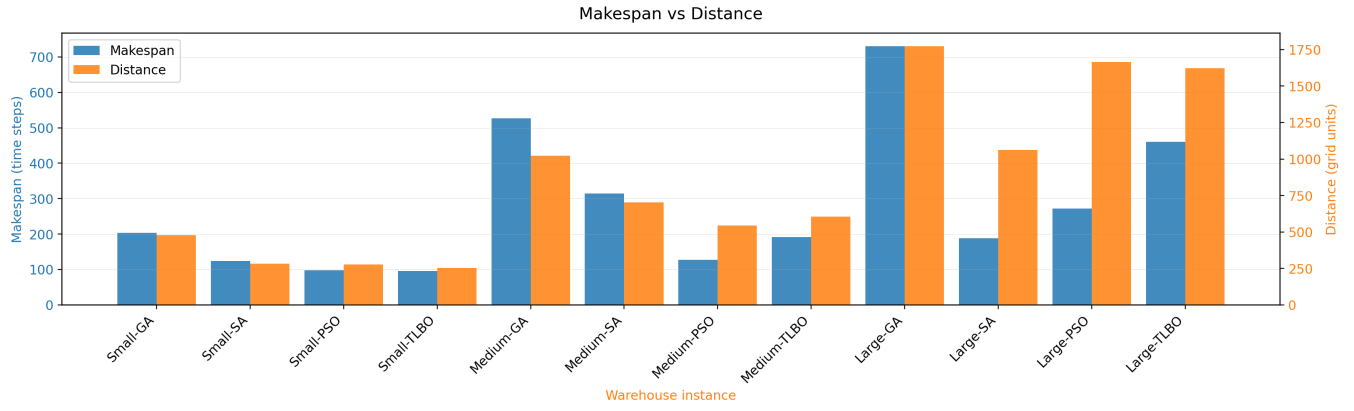


Fig. 2: Combined makespan and total travel distance across all instances.

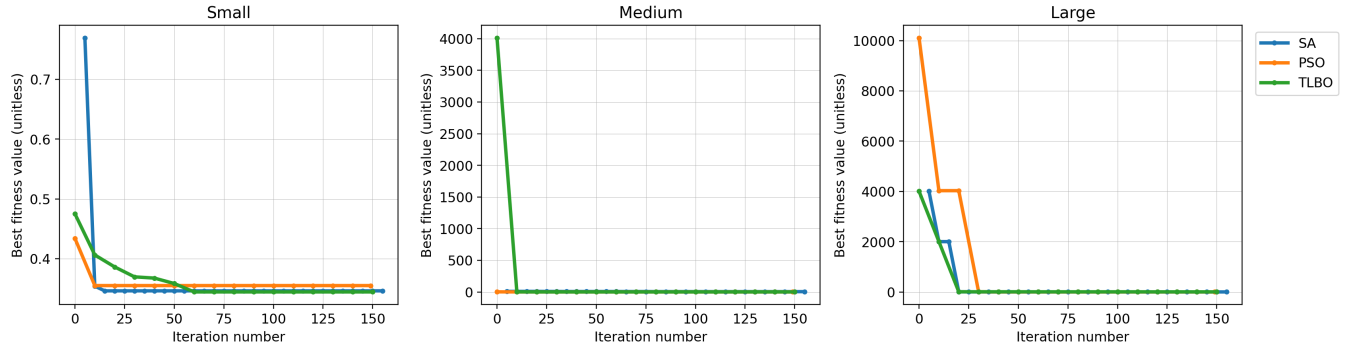


Fig. 3: Convergence behavior of SA, PSO, and TLBO across instances.

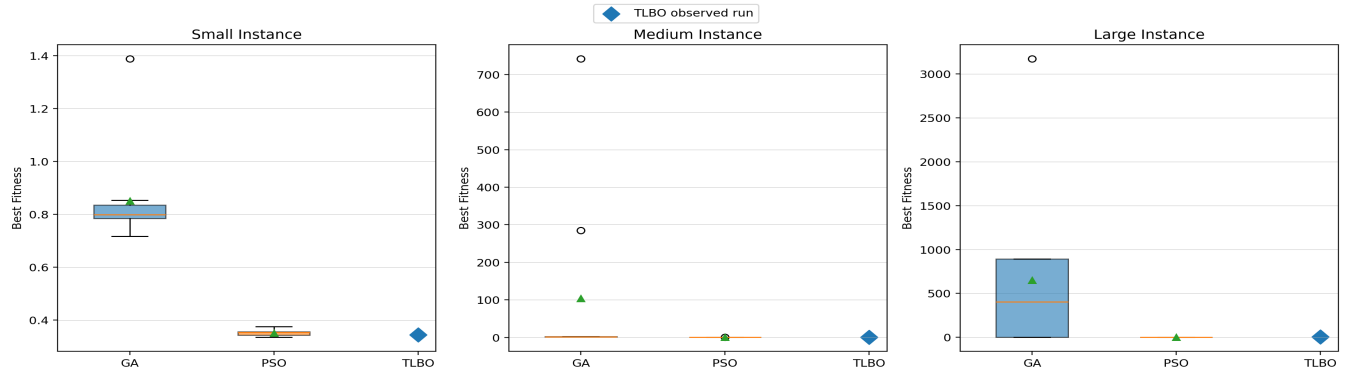


Fig. 4: Distribution of best fitness values across repeated runs for GA and PSO in the small, medium, and large instances. The available TLBO result for each instance is shown as a reference point.

- [4] L. Wang, M. Zhao, and Q. Li, "Optimal resource allocation and routing in robotic mobile fulfillment systems," *Computers and Operations Research*, vol. 168, p. 106750, 2024.
- [5] E.-G. Talbi, *Metaheuristics: From Design to Implementation*. Springer, 2021.
- [6] D. Boussa and M. Hamdi, "Simulated annealing based optimization for complex combinatorial problems: Recent advances," *Applied Soft Computing*, vol. 120, p. 108640, 2022.
- [7] J. Zhang and X. Wang, "Recent advances in particle swarm optimization and its applications," *Swarm and Evolutionary Computation*, vol. 75, p. 101202, 2023.
- [8] R. V. Rao, V. J. Savsani, and D. P. Vakharia, "Teaching-learning-based optimization: A novel method for constrained mechanical design optimization problems," *Computer-Aided Design*, vol. 43, no. 3, pp. 303–315, 2011.
- [9] M. H. Sulaiman, Z. Mustaffa, and M. I. M. Rashid, "An application of

teaching-learning-based optimization for solving the optimal power flow problem with stochastic wind and solar power generators," *Results in Control and Optimization*, vol. 10, p. 100187, 2023.